NIFES NISAB 2.0

CSS Practical Lesson

From Plain HTML to a Fully Styled Website

A Step-by-Step Guide for Beginners

■ Instructor Edition

Table of Contents

Introduction

What Is CSS?

CSS stands for **Cascading Style Sheets**. While HTML gives a page its *structure* (headings, paragraphs, buttons), CSS controls how everything *looks* — colours, fonts, spacing, layout, and animations.

Think of HTML as the skeleton of a building and CSS as the paint, furniture, and lighting.

How CSS Is Linked to HTML

We connect a CSS file to our HTML using a <link> tag inside <head>:

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

Both files must be in the same folder.

Three Ways to Apply CSS

- **Inline** — style attribute directly on a tag (not recommended for big projects)
- **Internal** — <style> tag inside <head> (useful for quick tests)
- **External** — a separate .css file linked to HTML **(best practice — what we use)**

The CSS Syntax

```
selector {
    property: value;
}
```

The **selector** targets an HTML element. The **property** is what you want to change. The **value** is what you change it to.

■ **Tip:** Every declaration ends with a semicolon (;). Forgetting it is the #1 beginner mistake!

STEP 1

See the Finished Design First

Before writing a single line of CSS, always look at what you are building. This gives you a mental map and helps you plan which elements need styling.

The NISAB 2.0 finished page has:

- A dark navy navbar with a logo, links, and a teal button
- A full-width hero section with a background image and dark overlay
- A centred About section on a light background
- Six skill cards arranged in a 3-column grid
- A dark Benefits section with a bulleted list
- A 3-column dark footer

Colour Palette

Name	Hex Code	Used For
Dark Navy	#0d1b2a	Navbar, Benefits bg, Footer bg
Teal Accent	#00b4d8	Buttons, hover links, accents
Light Gray	#f5f7fb	Page background
White	#ffffff	Cards, text on dark bg
Dark Text	#222222	Body text

■ **Tip:** Save this colour palette. You will use these exact hex codes throughout the lesson.

STEP 2

Assigning Classes and IDs to the HTML

Why Do We Need Classes and IDs?

CSS needs a way to *target* specific elements. Without classes or IDs, writing **h2 { color: red }** would turn *every* h2 on the page red. Classes and IDs let us be precise.

Classes vs IDs — The Rule

Feature	Class (.name)	ID (#name)
Usage	Many elements can share it	Only ONE element per page
HTML syntax	class="navbar"	id="hero"
CSS syntax	.navbar { }	#hero { }
Best for	Reusable styles (cards, buttons)	Unique sections (used rarely in CSS)

Your Task: Add These Classes to the HTML

HTML Element	Add This Class	Why
<header>	class="navbar"	Styles the top navigation bar
<nav>	class="nav-container"	Controls flex layout inside nav
<h1> (logo)	class="logo"	White text styling for the logo
(nav links)	class="nav-links"	Horizontal flex list with gaps
Navbar <button>	class="btn"	Reusable teal button style
1st <section>	class="hero"	Full-width hero image section
Buttons div in hero	class="hero-buttons"	Flex row for two buttons
Features div in hero	class="hero-features"	4-column grid of feature badges
Learn More button	class="btn-outline"	Ghost/outline button variant
2nd <section>	class="about"	Centred about section
3rd <section>	class="tracks"	Skill tracks section
Wrapper div in tracks	class="track-container"	3-column CSS grid container
Each skill div	class="card"	White card with shadow & hover

4th <section>	class="benefits"	Dark benefits section
<footer>	class="footer"	Dark footer background
Footer columns div	class="footer-grid"	3-column grid in footer
Copyright <p>	class="copyright"	Centred copyright line

How to Add a Class — Example

Before (raw HTML):

```
<header>
```

After (with class):

```
<header class="navbar">
```

■ **Tip:** Use only lowercase letters and hyphens in class names. Never use spaces — that creates two classes.

■ Open your raw HTML file

■ Add every class from the table above

■ Save the file — it won't look different yet. That's normal!

STEP 3

CSS Reset & Body Styles

Why We Need a Reset

Every browser (Chrome, Firefox, Safari) applies its own default margin and padding to elements. This causes pages to look slightly different in each browser. A CSS reset removes all of these defaults so we start from a clean slate.

Create your style.css file and type this first:

```
/* RESET */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

Breaking It Down

Property	Value	What It Does
*	(selector)	Targets EVERY element on the page
margin	0	Removes all default outer spacing
padding	0	Removes all default inner spacing
box-sizing	border-box	Padding is included inside the element's total width

Body Styles

Now add the body block right after the reset:

```
/* BODY */
body {
    font-family: Arial, Helvetica, sans-serif;
    line-height: 1.6;
    background: #f5f7fb;
    color: #222;
}
```

- **font-family**: Sets the font for the whole page. We list backups in case Arial isn't available.
- **line-height: 1.6**: Adds vertical breathing room between lines of text.
- **background: #f5f7fb**: Very light blue-grey page background.
- **color: #222**: Near-black default text colour.

■ **Tip:** font-family, background, and color set on body are *inherited* by all child elements — you don't need to repeat them everywhere.

■ Create an empty file called style.css in the same folder as your HTML

■ Add the reset block

■ Add the body block

■ Save and open your HTML in a browser — the font should change

STEP 4

Styling the Navbar

Concept: Flexbox

Flexbox is a CSS layout tool that arranges items in a row (or column). Setting **display: flex** on a container turns its direct children into flex items that can be aligned, spaced, and sized together.

Add this to style.css:

```
/* NAVBAR */
.navbar {
    background: #0d1b2a;
    padding: 20px 8%;
}

.nav-container {
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.logo {
    color: white;
}

.nav-links {
    display: flex;
    gap: 25px;
    list-style: none;
}

.nav-links a {
    text-decoration: none;
    color: white;
    font-size: 14px;
}

.nav-links a:hover {
    color: #00b4d8;
}
```

Property Glossary for This Step

Property	Value Used	What It Does
background	#0d1b2a	Dark navy background on the navbar

padding	20px 8%	20px top/bottom, 8% left/right — creates breathing room
display	flex	Enables flexbox on .nav-container
justify-content	space-between	Pushes logo left, links centre, button right
align-items	center	Vertically centres all items
gap	25px	Space between each nav link
list-style	none	Removes the bullet points from the
text-decoration	none	Removes the underline from <a> tags
:hover	(pseudo-class)	Applies styles when the user mouses over

■ **Tip:** justify-content controls the MAIN axis (horizontal by default). align-items controls the CROSS axis (vertical).

■ Add all navbar styles to style.css

■ Save and refresh — your navbar should be dark with a teal hover on links

■ Try changing justify-content to flex-start and see what happens

STEP 5

Styling the Buttons

We have two button types: a filled teal **.btn** and a transparent **.btn-outline**. Both share the same padding and border-radius, only the background and border differ. This is the power of reusable classes.

```
/* BUTTONS */
.btn {
    background: #00b4d8;
    color: white;
    border: none;
    padding: 12px 20px;
    border-radius: 5px;
    cursor: pointer;
}
.btn:hover {
    background: #0096c7;
}
.btn-outline {
    background: transparent;
    border: 2px solid white;
    color: white;
    padding: 12px 20px;
    border-radius: 5px;
    cursor: pointer;
}
```

New Properties

- **border: none** — removes the default grey border browsers add to buttons
- **border-radius: 5px** — rounds the corners slightly
- **cursor: pointer** — shows the hand cursor when hovering, signalling it's clickable
- **background: transparent** — makes the button see-through (shows the hero image behind)

Understanding Padding Shorthand

padding: 12px 20px means:

- First value (12px) = top & bottom padding
- Second value (20px) = left & right padding

■ **Tip:** You can use padding: 12px 20px 12px 20px (top, right, bottom, left — clockwise) for full control.

■ Add both button styles

■ Hover over the Register Now button — it should darken slightly

■ Notice how both buttons share the same padding but look different

STEP 6

Styling the Hero Section

Concept: Background Images & Overlays

A background image alone can make text hard to read. We fix this by adding a dark semi-transparent overlay using a CSS gradient layered on top of the image.

```
/* HERO */
.hero {
    background: linear-gradient(rgba(0,0,0,0.6), rgba(0,0,0,0.6)),
    url('https://images.unsplash.com/photo-1522202176988-66273c2fd55f');
    background-size: cover;
    background-position: center;
    color: white;
    text-align: center;
    padding: 120px 10%;
}

.hero h2 {
    font-size: 55px;
    margin-bottom: 20px;
}

.hero p {
    max-width: 700px;
    margin: auto;
    margin-bottom: 30px;
}

.hero-buttons {
    display: flex;
    justify-content: center;
    gap: 20px;
    margin-bottom: 40px;
}

.hero-features {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 20px;
}

.hero-features div {
    background: rgba(255,255,255,0.1);
    padding: 20px;
```

```
    border-radius: 10px;
}
```

Key Concepts Explained

linear-gradient(rgba(0,0,0,0.6), rgba(0,0,0,0.6)) — creates a dark overlay. rgba(0,0,0,0.6) is black at 60% opacity. Placing it before the URL layers it on top of the image.

background-size: cover — stretches the image to fill the entire section without leaving gaps.

max-width: 700px; margin: auto — limits the paragraph width to 700px and centres it horizontally.

grid-template-columns: repeat(4, 1fr) — creates 4 equal columns. **1fr** means 'one fraction of available space'.

rgba(255,255,255,0.1) — white at 10% opacity, creating a frosted glass look on the feature badges.

■ **Tip:** repeat(4, 1fr) is shorthand for 1fr 1fr 1fr 1fr — saves typing and is easier to change.

■ Add all hero styles

■ Refresh — you should see the background image with a dark overlay

■ Try changing the 0.6 in rgba to 0.2 to see a lighter overlay

■ Change repeat(4, 1fr) to repeat(2, 1fr) and see the layout shift

STEP 7

Styling the About Section

The About section is simple — it uses padding, centred text, and a font-size on the heading. This step reinforces spacing concepts.

```
/* ABOUT */
.about {
    padding: 80px 10%;
    text-align: center;
}

.about h2 {
    margin-bottom: 20px;
    font-size: 40px;
}
```

The Box Model

Every HTML element is a rectangular box made of four layers from inside out:

Layer	Property	Description
Content	width / height	The actual text or image
Padding	padding	Space inside the border
Border	border	The visible outline around the box
Margin	margin	Space outside the border

■ **Tip:** Use your browser's DevTools (F12 → Elements) to see the Box Model visually for any element.

■ Add the About section styles

■ Try changing padding: 80px 10% to padding: 20px 10% — notice how squashed it becomes

■ Restore it to 80px when done

STEP 8

Skill Track Cards — CSS Grid & Hover Effects

Concept: CSS Grid

CSS Grid is a two-dimensional layout system. Unlike Flexbox (one row or column), Grid lets you define both rows and columns. We use it here to create a 3-column layout of skill cards.

```
/* TRACKS */
.tracks {
    padding: 80px 10%;
}

.tracks h2 {
    text-align: center;
    margin-bottom: 50px;
    font-size: 40px;
}

.track-container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 25px;
}

.card {
    background: white;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 5px 15px rgba(0,0,0,0.1);
    transition: 0.3s;
}

.card:hover {
    transform: translateY(-10px);
}
```

Understanding box-shadow

box-shadow: 0 5px 15px rgba(0,0,0,0.1) has four parts:

- **0** — horizontal offset (no shadow to the left or right)
- **5px** — vertical offset (shadow 5px below the card)
- **15px** — blur radius (how soft/blurry the shadow is)
- **rgba(0,0,0,0.1)** — black at 10% opacity (very subtle)

Understanding transition & transform

transition: 0.3s tells the browser to animate any property change over 0.3 seconds. Without it, the hover effect would be instant and jarring.

transform: translateY(-10px) moves the card 10px *upward* on hover, creating a 'lift' effect. Negative Y = up.

■ **Tip:** transition must be on the base element (.card), not on .card:hover, so it also animates back smoothly when you mouse out.

■ Add all track and card styles

■ Hover over a card — it should lift upward

■ Try box-shadow: 0 10px 30px rgba(0,180,216,0.3) for a teal glow

■ Try translateY(-20px) for a more dramatic lift

STEP 9

Benefits Section — Dark Background & Lists

The Benefits section uses the same dark navy as the navbar to create visual contrast against the light sections around it.

```
/* BENEFITS */
.benefits {
    background: #0d1b2a;
    color: white;
    padding: 80px 10%;
}

.benefits h2 {
    margin-bottom: 20px;
}

.benefits ul {
    padding-left: 20px;
}
```

Alternating Dark & Light Sections

Good web design uses alternating light and dark sections to create visual rhythm. Looking at our page so far:

- Navbar — Dark
- Hero — Dark (image overlay)
- About — Light (#f5f7fb)
- Tracks — Light (#f5f7fb)
- Benefits — Dark (#0d1b2a) ← this step
- Footer — Dark (#111)

■ **Tip:** Inheriting color: white on .benefits means all text inside it (including li items) becomes white automatically.

■ Add the benefits styles

■ Notice how color: white on the parent automatically colours the list items

■ Try removing padding-left: 20px to see the bullets disappear off-screen

STEP 10

Styling the Footer

The footer uses a three-column grid layout with a very dark background. This is the same grid concept as the track cards, just with different content.

```
/* FOOTER */
.footer {
    background: #111;
    color: white;
    padding: 60px 10%;
}
.footer-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 40px;
}
.copyright {
    margin-top: 40px;
    text-align: center;
}
```

Flexbox vs Grid — When to Use Which?

Use Flexbox when…	Use Grid when…
Layout is one-dimensional (row OR column)	Layout is two-dimensional (rows AND columns)
Items should size themselves naturally	You need precise column/row control
Navbar, button groups, hero buttons	Cards, footer columns, feature grids

■ **Tip:** gap works identically in both Flexbox and Grid — it sets the space between items.

■ Add all footer styles

■ Your page is now fully styled on desktop!

■ Try changing repeat(3, 1fr) to repeat(2, 1fr) to see two columns

STEP 11

Responsive Design — @media Queries

What Is Responsive Design?

Responsive design means the page looks good on all screen sizes — desktop, tablet, and mobile. CSS media queries let you apply different styles below certain widths.

Syntax of a Media Query

```
@media (max-width: 900px) {
    /* Styles inside here ONLY apply when
       the screen is 900px wide or less */
}
```

Add This at the Very End of style.css

```
/* RESPONSIVE */
@media (max-width: 900px) {

    .nav-container {
        flex-direction: column;
        gap: 20px;
    }

    .nav-links {
        flex-wrap: wrap;
        justify-content: center;
    }

    .hero h2 {
        font-size: 38px;
    }

    .hero-features {
        grid-template-columns: 1fr 1fr;
    }

    .track-container {
        grid-template-columns: 1fr;
    }

    .footer-grid {
        grid-template-columns: 1fr;
    }
}
```

What Each Rule Does

• **flex-direction: column** — stacks the logo, links, and button vertically on small screens

• **flex-wrap: wrap** — allows nav links to wrap to a second line if needed

• **font-size: 38px** — reduces the hero heading so it doesn't overflow on mobile

• **1fr 1fr** — changes feature grid to 2 columns instead of 4

• **1fr** — changes cards and footer to a single column for easy scrolling

■ **Tip:** Test your responsive styles by resizing the browser window or pressing F12 → Toggle Device Toolbar in Chrome.

■ Add the media query block

■ Open DevTools (F12) and click the mobile icon

■ Select iPhone SE or similar — the layout should stack

■ Congratulations! Your page is now fully styled and responsive ■

CSS Cheat Sheet — All Properties Used

Property	Example Value	What It Does
background	#0d1b2a	Sets background colour or image
color	white	Sets text colour
font-family	Arial, sans-serif	Sets the font
font-size	55px	Sets text size
line-height	1.6	Spacing between lines
padding	20px 8%	Space inside element
margin	0 auto	Space outside element / centering
border	2px solid white	Border around element
border-radius	10px	Rounds corners
border-none	none	Removes border
display	flex / grid	Enables Flexbox or Grid
justify-content	space-between / center	Aligns items on main axis
align-items	center	Aligns items on cross axis
gap	25px	Space between flex/grid items
flex-direction	column	Stacks flex items vertically
flex-wrap	wrap	Allows flex items to wrap
grid-template-cols	repeat(3, 1fr)	Defines grid columns
list-style	none	Removes bullet points
text-decoration	none	Removes underline from links
text-align	center	Aligns text horizontally
max-width	700px	Limits element's maximum width
box-shadow	0 5px 15px rgba(0,0,0,.1)	Adds shadow under element
transition	0.3s	Animates property changes
transform	translateY(-10px)	Moves/rotates/scales element
cursor	pointer	Shows hand cursor on hover
background-size	cover	Stretches image to fill element
background-position	center	Positions background image

@media	(max-width: 900px)	Applies styles at certain screen sizes
:hover	(pseudo-class)	Styles applied on mouse hover
rgba()	rgba(0,0,0,0.6)	Colour with opacity (0=transparent, 1=solid)

■ **Well done!** You have gone from a completely unstyled HTML page to a fully designed, responsive website using real CSS techniques used by professional developers every day. Keep experimenting — try changing colours, font sizes, and layouts to make the design your own.